

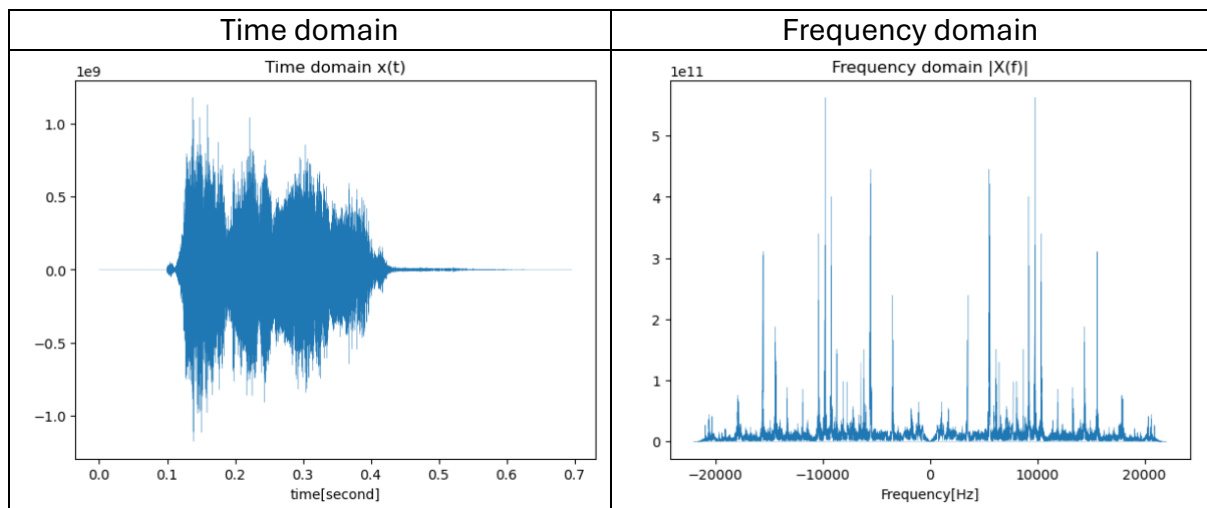
All required files and parameters have been sent to your Chula email.

1. (15 points) Audio Analysis

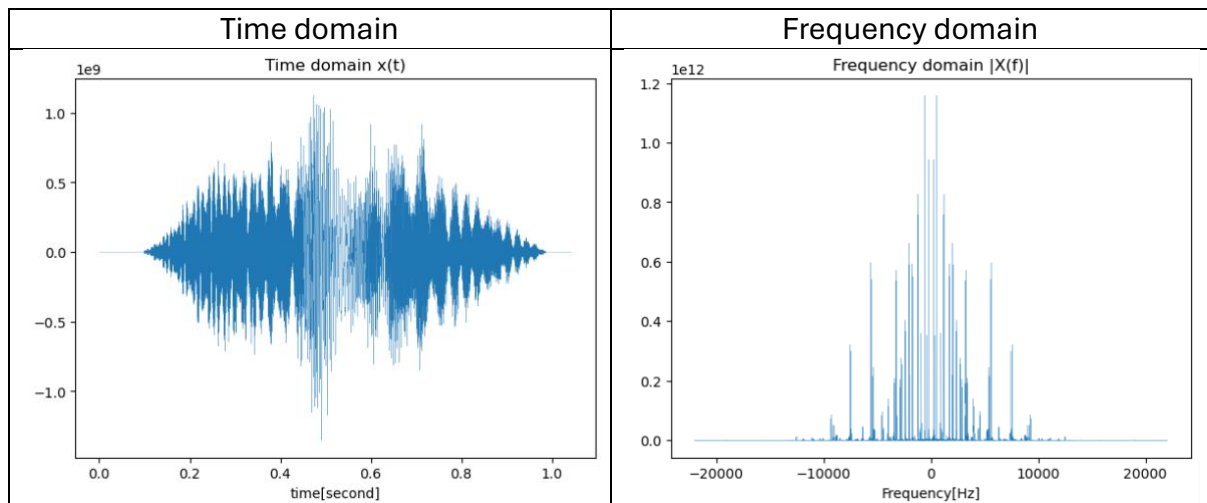
Download **file1.wav** and **file2.wav** from the links provided in your email. Perform a DFT to analyze and compare the signals.

1.1) Plot the **file1** and **file2** signals in time domain and its frequency spectrum (low frequency at the center)

file1:



file2:



2.2) Determine top three dominant frequencies of each signal in Hz. **EXCLUDING** the dc component and those that are the consequence of the **complex conjugate symmetry** property. Explain how you obtain these values.

Note: The 1st, 2nd, and 3rd dominant frequencies are those with the highest, second highest, and third highest magnitudes in the spectrum.

File1:

1st dominant frequency : ____ 9780.04495992702 ____ Hz

2nd dominant frequency : ____ 9781.481722812276 ____ Hz

3rd dominant frequency : ____ 5517.169479377077 ____ Hz

file2:

1st dominant frequency : ____ 556.5182450043441 ____ Hz

2nd dominant frequency : ____ 557.4761077324066 ____ Hz

3rd dominant frequency : ____ 547.897480451781 ____ Hz

Explanation:

```
# Frequency domain
dfted_x = np.abs( fft.fft(x_mono) ) # np.abs for imaginary number is a**2 + b**2 for imaginary number a+jb
freq = fft.fftfreq(len(x_mono), d=1/sr) # d : Sample spacing (inverse of the sampling rate)

dfted_x[0] = 0 # exclude DC component

sorted_index = dfted_x.argsort()[::-1] # sort index by magnitude (descending order)

print(*np.abs(freq[sorted_index][:6:2]), sep='\n')
# Top 3 highest dominant frequencies (due to complex conjugate symmetry, I need to set step=2, i.e, top magnitude of order 2*i == top magnitude of order 2*i+1)
# The reason I put np.abs because if the value(magnitude) is equal, so the sorting algorithm randomly put either f$ or -$ first and the other second.

print(*dfted_x[sorted_index][:6:2], sep='\n') # print correspond magnitude
```

2.3) Analyze and compare the frequencies of the two signals and their corresponding sounds in your own words.

By the observation of frequencies plots of file1 and file 2. You'll see that file1 has a high magnitude in the high frequencies area so file1 a **treble** sound. On the other hand, file2 has a high magnitude in the low frequencies area that made file2 a **bass** sound.

Note:

-If you use python, you can use “`scipy.io.wavfile.read`” for reading WAV file,

see <https://docs.scipy.org/doc/scipy/reference/generated/scipy.io.wavfile.read.html>

2) (15 points)

Hamtaro plans to design a system where the relationship between the input $x[n]$ and the output $y[n]$ is defined as follows:

$$y[n] = a_1x[n] + a_2x[n + n_1] + a_3x[n + n_2]$$

The parameters a_1, a_2, a_3, n_1 , and n_2 are defined in params.json. Provide the values below:

$a_1 =$	4	$a_2 =$	-2	$a_3 =$	-2	$n_1 =$	-1	$n_2 =$	-2
---------	---	---------	----	---------	----	---------	----	---------	----

2.1) Determine the impulse response $h[n]$ and frequency response $H(e^{j\omega})$ of the system and demonstrate the solution steps in your own words.

Find $h[n]$

$$y[n] = a_1x[n] + a_2x[n + n_1] + a_3x[n + n_2]$$

$$y[n] = 4x[n] - 2x[n - 1] - 2x[n - 2]$$

$$x * h[n] = 4x[n] - 2x[n - 1] - 2x[n - 2]$$

$$x * h[n] = \sum_{k=-\infty}^{\infty} x[n - k]h[k]$$

$$\therefore h[n] = \begin{cases} 4, & \text{if } n = 0 \\ -2, & \text{if } n = 1 \\ -2, & \text{if } n = 2 \\ 0, & \text{otherwise} \end{cases}$$

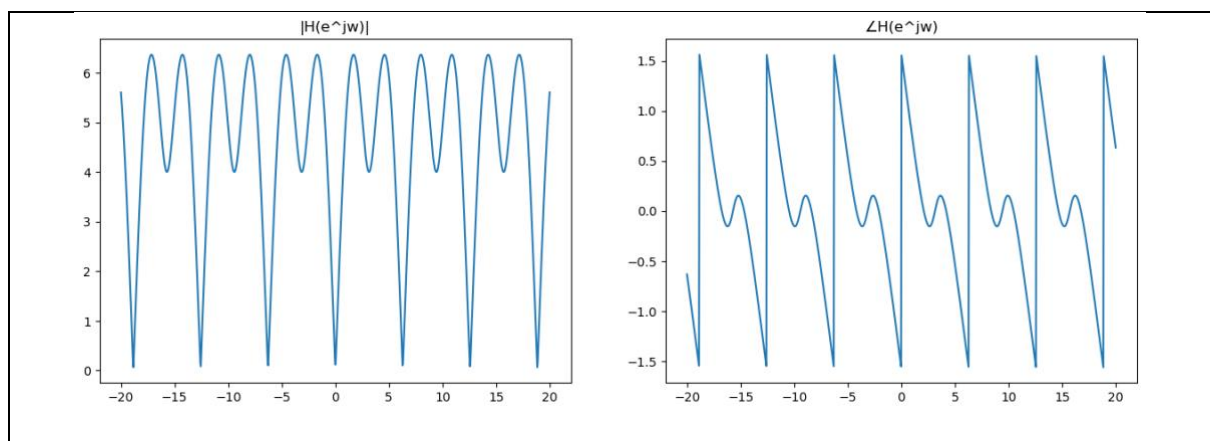
find $H(e^{j\omega})$

$$H(e^{j\omega}) = \sum_{n=-\infty}^{\infty} h[n]e^{-j\omega n}$$

$$= h[0]e^0 + h[1]e^{-j\omega} + h[2]e^{-2j\omega}$$

$$= 4 - 2e^{-j\omega} - 2e^{-2j\omega}$$

2.2) Plot magnitude and phase of the frequency response.



2.3) Determine and plot the DFT of the system output using your student id as the input sequence $x[n]$ to the system. For example, if your student ID is 6530000021, the input sequence $x[n] = \{6, 5, 3, 0, 0, 0, 0, 2, 1\}$. Finally, show the solution steps in your own words.

2.3

Student ID: 6532136721

Convolution

	6	5	3	2	1	3	6	7	2	1
								4	-2	-2
	-12	-10	-6	-4	-2	-6	-12	-14	-4	-2
	-12	-10	-6	-4	-2	-6	-12	-14	-4	-2
24	20	12	8	4	12	24	28	8	4	
24	8	-10	-8	-6	6	16	10	-18	-14	-6
										-2

$y = \{24, 8, -10, -8, -6, 6, 16, 10, -18, -14, -6, -2\}$

Find DFT

$$Y[k] = \sum_{n=0}^{11} y[n] e^{-j \frac{2\pi}{12} nk}$$

```
x = [6, 5, 3, 2, 1, 3, 6, 7, 2, 1]
h = [4, -2, -2]

y = np.convolve(x, h)
print(y.tolist())

dft_y = np.array([
    sum( yn * np.exp(-1j * 2 * np.pi * n * k / len(y)) for n, yn in enumerate(y) )
    for k in range(len(y))
])
print(dft_y)

freq_fft = fft.fft(x, n=12)*fft.fft(h, n=12)
assert np.all( np.abs( dft_y - freq_fft ) < 1e-12 ) # Convolution check

_, ax = plt.subplots(nrows=1, ncols=2, figsize=(15, 5))

ax[0].set_title('|Y[k]|')
ax[1].set_title('<Y[k]')

mag = np.abs(dft_y)
phase = np.angle(dft_y)

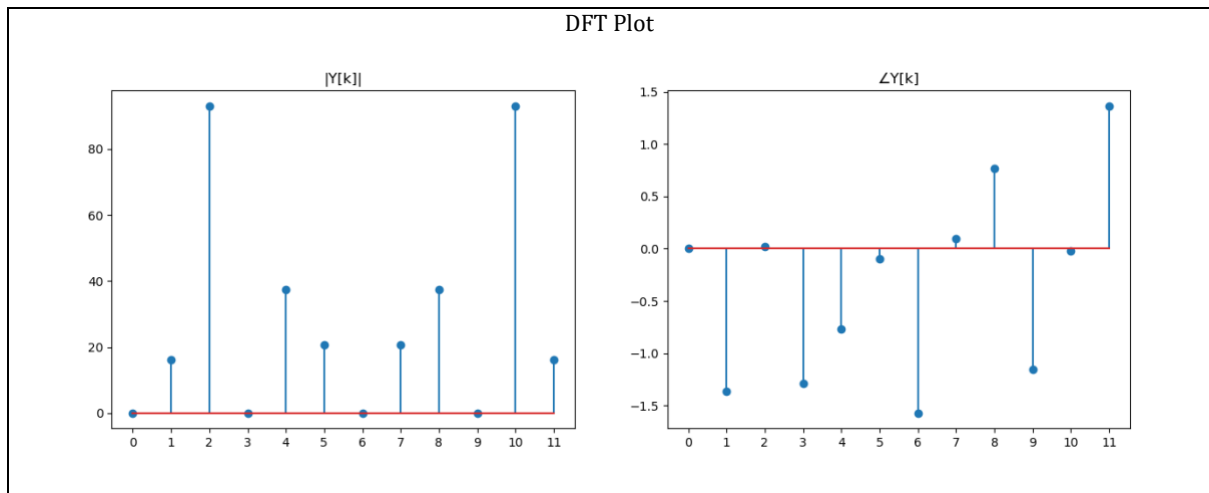
ax[0].stem(mag)
ax[0].set_xticks(range(len(mag)))
ax[1].stem(phase)
ax[1].set_xticks(range(len(phase)))

plt.show()
✓ 0.0s
```

The output from the cell:

[24, 8, -10, -8, -6, 6, 16, 10, -18, -14, -6, -2]

[0.00000000e+00+0.00000000e+00j 3.33974596e+00-1.59282032e+01j
9.30000000e+01+1.73205081e+00j 4.58933059e-16-1.55431223e-15j
2.70000000e+01-2.59807621e+01j 2.06602540e+01-2.07179677e+00j
0.00000000e+00-3.56458876e-14j 2.06602540e+01+2.07179677e+00j
2.70000000e+01+2.59807621e+01j 2.43803750e-14-5.46229728e-14j
9.30000000e+01-1.73205081e+00j 3.33974596e+00+1.59282032e+01j]



References

DFT(FFT) things

<https://numpy.org/doc/stable/reference/routines.fft.html>

Magnitude and Phase

<https://numpy.org/doc/stable/reference/generated/numpy.absolute.html>

<https://numpy.org/doc/stable/reference/generated/numpy.angle.html#numpy.angle>

Finding frequencies with correspond highest magnitude

<https://numpy.org/doc/stable/reference/generated/numpy.argsort.html>

Frequency Plot reference

<https://stackoverflow.com/questions/74063787/how-to-plot-frequency-data-from-a-wav-file-in-python>